Final Project

In [78]:
```
#import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

In [79]:
```
#load dataset
df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')
df
```

Out[79]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	E
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	
...	...	...	...	...	...	...	
1465	36	No	Travel_Frequently	884	Research & Development	23	
1466	39	No	Travel_Rarely	613	Research & Development	6	
1467	27	No	Travel_Rarely	155	Research & Development	4	
1468	49	No	Travel_Frequently	1023	Sales	2	
1469	34	No	Travel_Rarely	628	Research & Development	8	

1470 rows × 35 columns

In [80]:
```
#data cleaning
#looking for missing values
print('Missing values:')
print( df.isnull().sum())
```

```
Missing values:
Age                         0
Attrition                   0
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
```

In [81]:
```
#dropping duplicate values
df= df.drop_duplicates()
df
```

Out[81]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	E
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	
...	...	...	...	...	...	...	
1465	36	No	Travel_Frequently	884	Research & Development	23	
1466	39	No	Travel_Rarely	613	Research & Development	6	
1467	27	No	Travel_Rarely	155	Research & Development	4	
1468	49	No	Travel_Frequently	1023	Sales	2	
1469	34	No	Travel_Rarely	628	Research & Development	8	

1470 rows × 35 columns

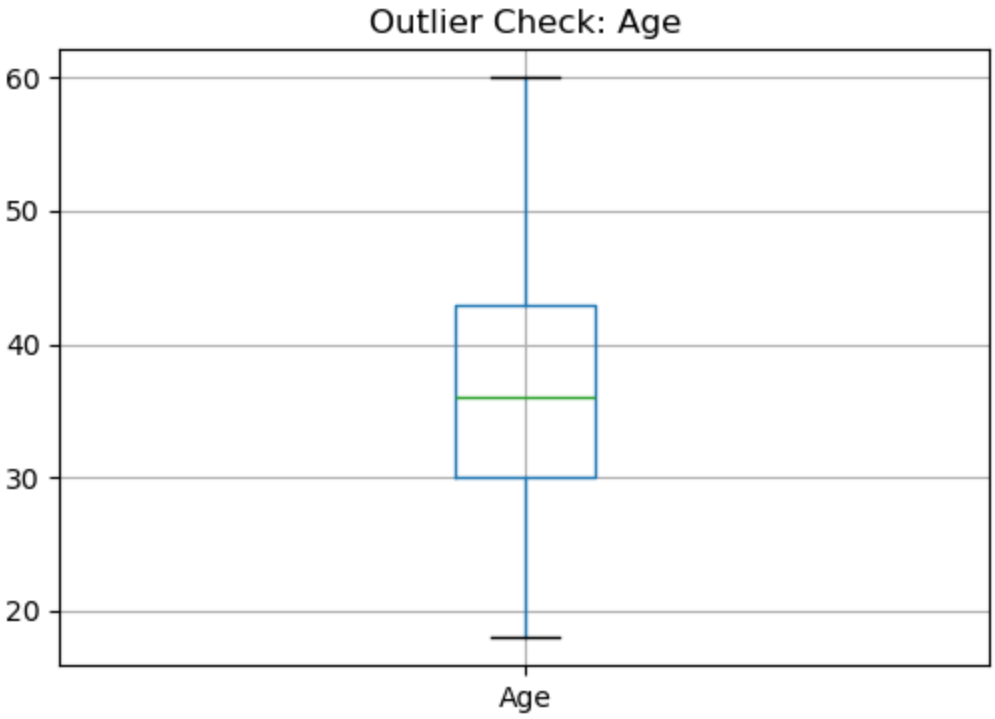
In [82]:
```
#dropping unnecessary columns
if 'EmployeeNumber' in df.columns:
 df = df.drop(columns=['EmployeeNumber'])
df
```

Out[82]:

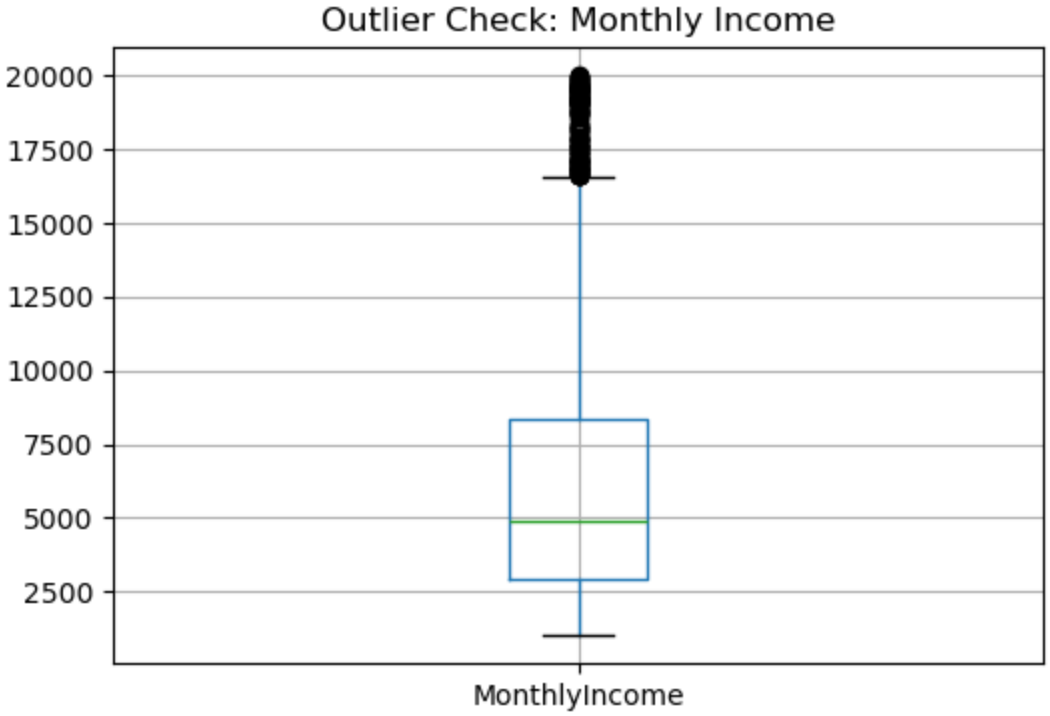
	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	E
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	
...	...	...	...	...	...	...	
1465	36	No	Travel_Frequently	884	Research & Development	23	
1466	39	No	Travel_Rarely	613	Research & Development	6	
1467	27	No	Travel_Rarely	155	Research & Development	4	
1468	49	No	Travel_Frequently	1023	Sales	2	
1469	34	No	Travel_Rarely	628	Research & Development	8	

1470 rows × 34 columns

In [83]:
```
#check for outliers
plt.figure(figsize=(6,4))
df.boxplot(column=['Age'])
plt.title('Outlier Check: Age')
plt.show()
```

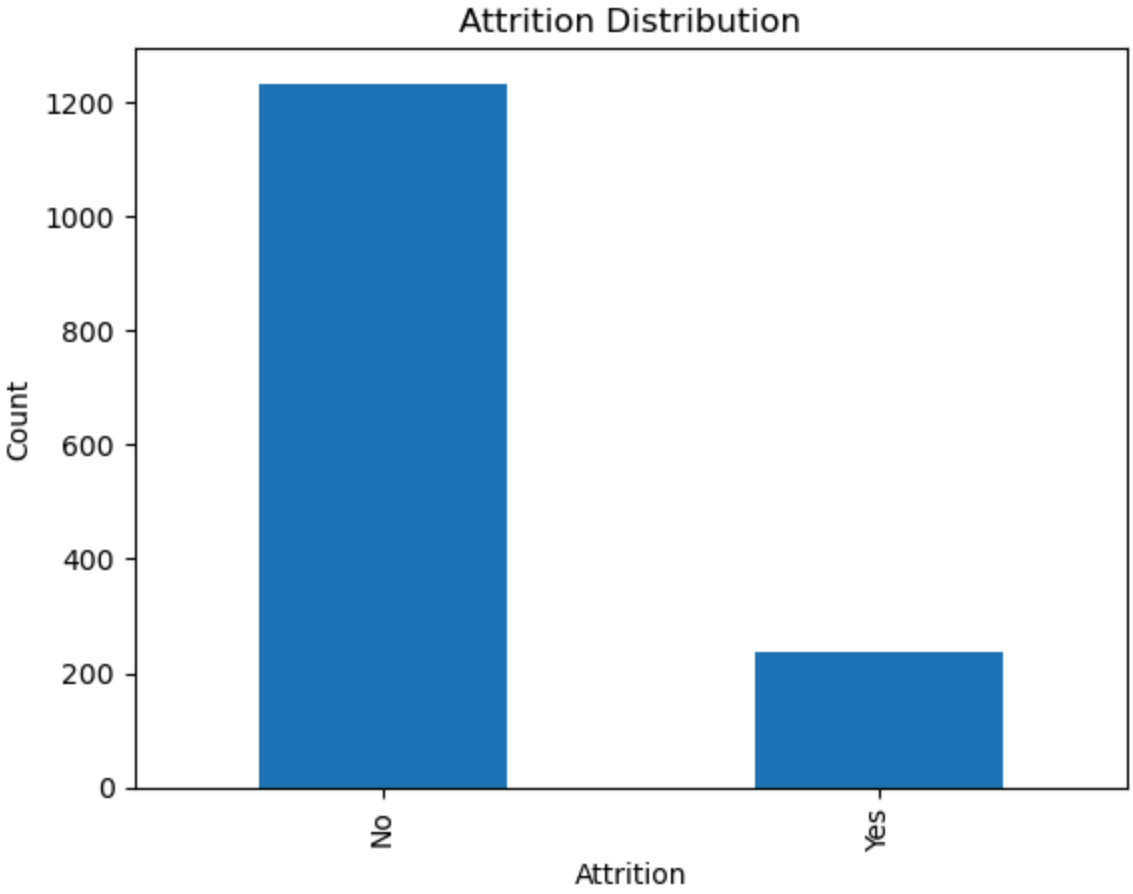


In [84]:
```
# check for outliers
plt.figure(figsize=(6,4))
df.boxplot(column=['MonthlyIncome'])
plt.title('Outlier Check: Monthly Income')
plt.show()
```



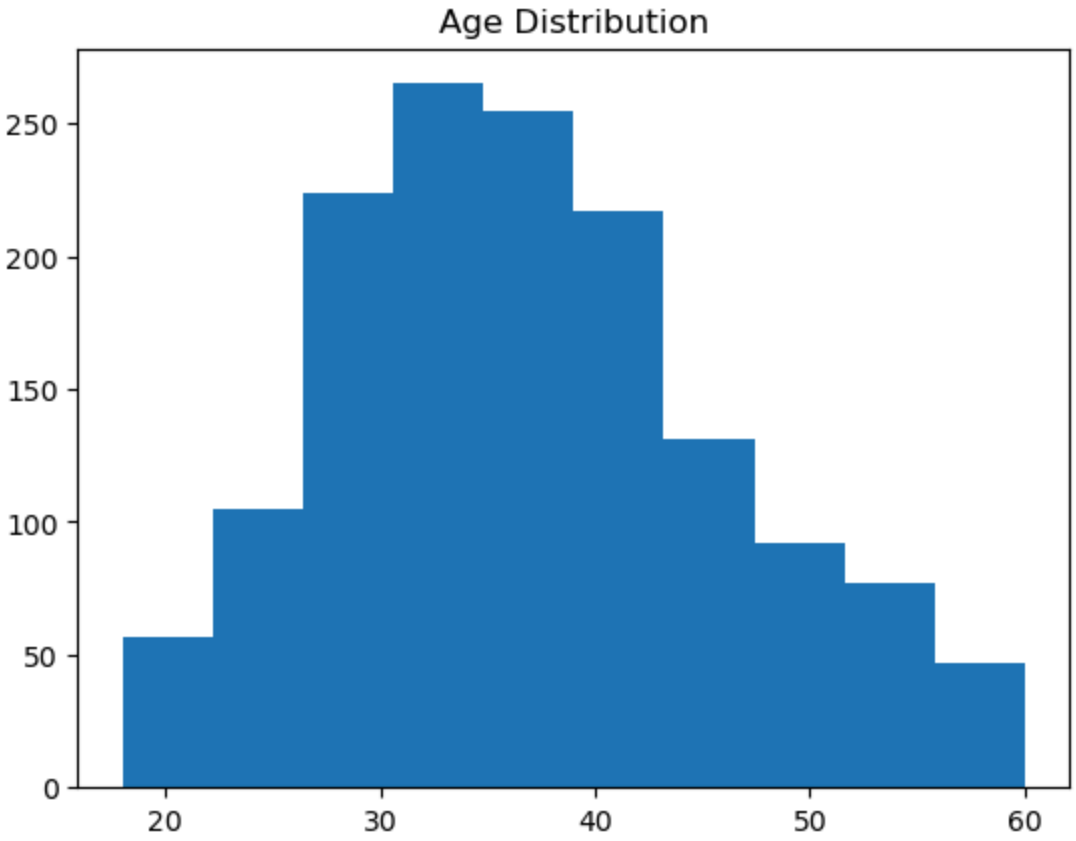
In [85]:
```
# Exploratory Data Analysis (EDA)
# Attrition distribution
df['Attrition'].value_counts().plot(kind='bar')
plt.title('Attrition Distribution')
```

```
plt.xlabel('Attrition')
plt.ylabel('Count')
plt.show()
```



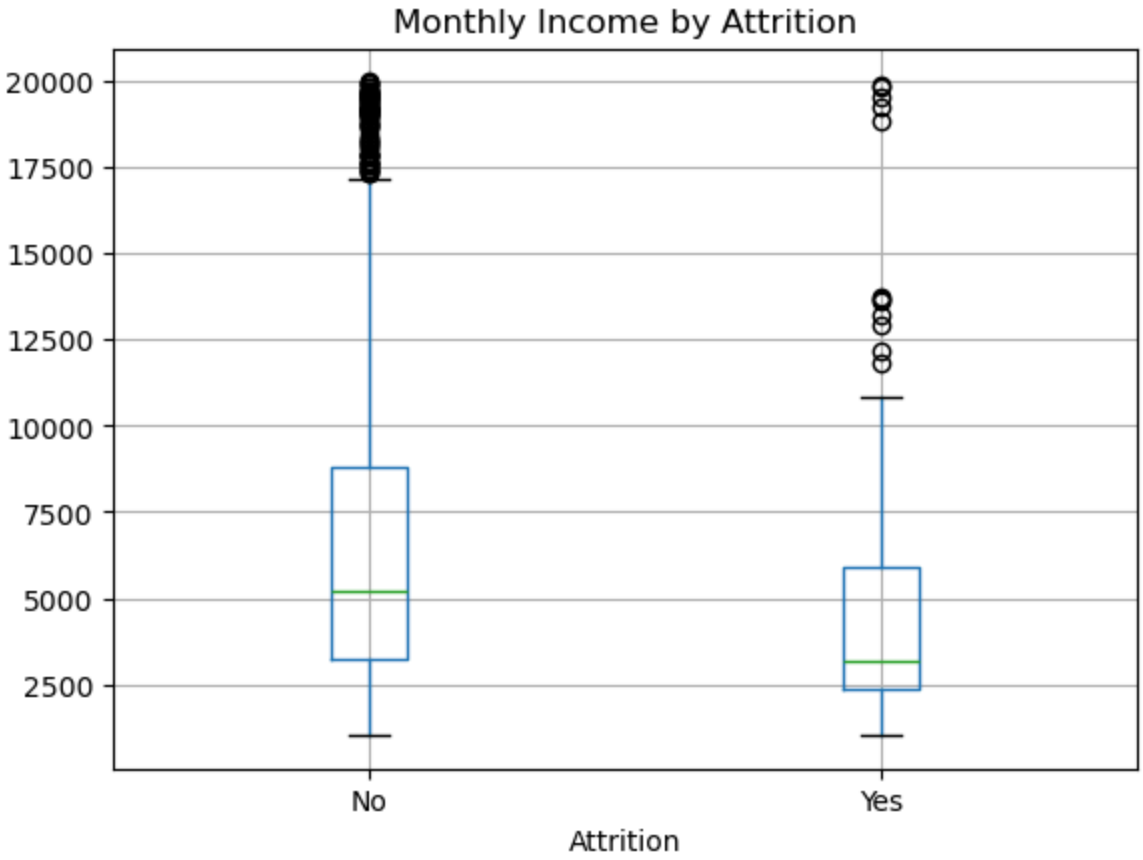
In [86]:
```
# Exploratory Data Analysis (EDA)

# Age distribution by using a histogram
plt.hist(df['Age'])
plt.title('Age Distribution')
plt.show()
```



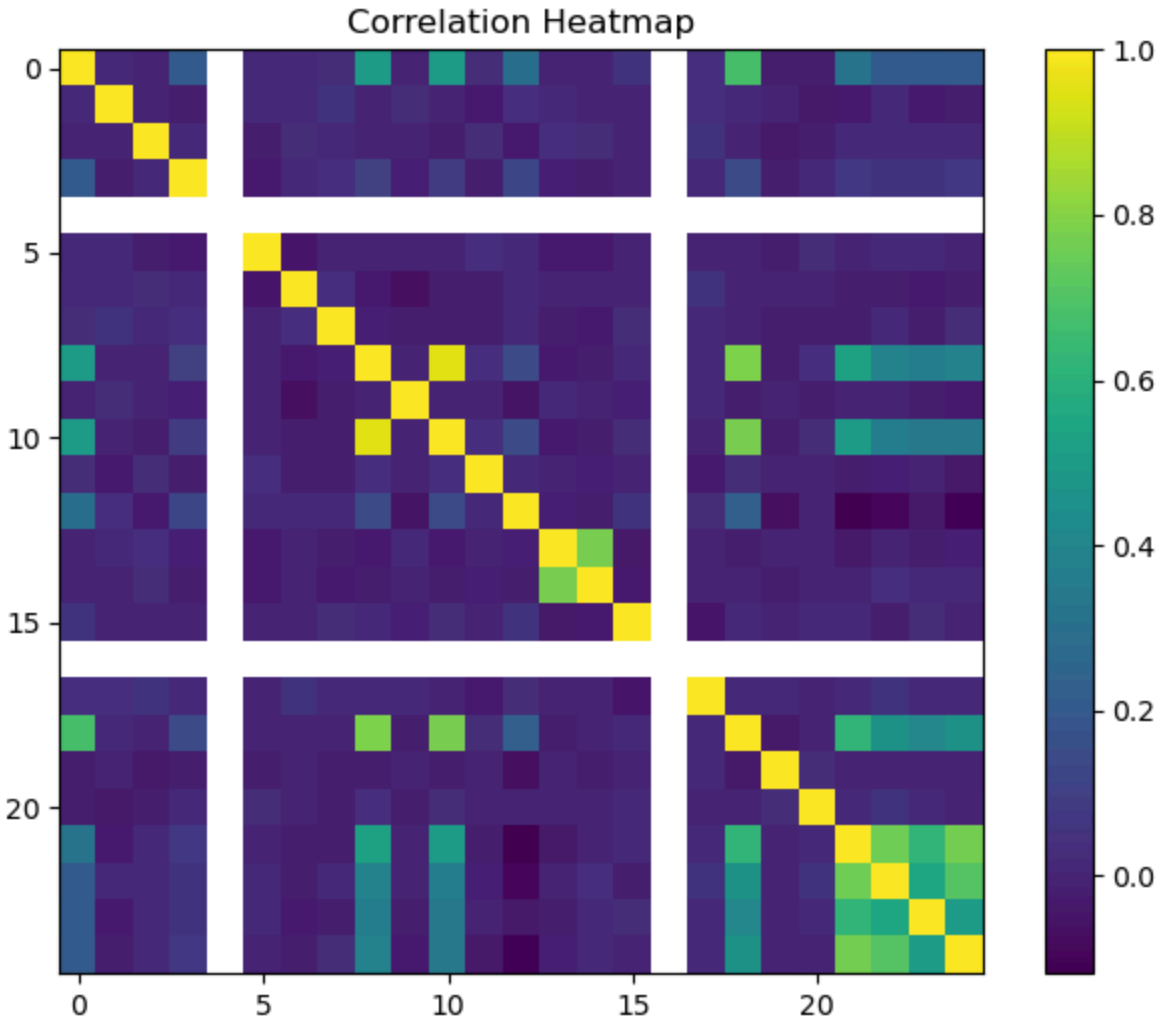
In [87]:
```
# Exploratory Data Analysis (EDA)

# Monthly income vs Attrition
df.boxplot(column='MonthlyIncome', by='Attrition')
plt.title('Monthly Income by Attrition')
plt.suptitle('')
plt.show()
```



In [88]:
```
# Exploratory Data Analysis (EDA)

# Correlation heatmap
numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()
plt.figure(figsize=(8,6))
plt.imshow(corr)
plt.title('Correlation Heatmap')
plt.colorbar()
plt.show()
```



In [89]:
```
# Tranforming variables
# Encoding categorical values
le = LabelEncoder()
for col in df.select_dtypes(include=['object']).columns:
 df[col] = le.fit_transform(df[col])
df
```

Out[89]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Edu
0	41	1	2	1102	2	1	
1	49	0	1	279	1	8	
2	37	1	2	1373	1	2	
3	33	0	1	1392	1	3	
4	27	0	2	591	1	2	
...	...	...	...	...	...	...	
1465	36	0	1	884	1	23	
1466	39	0	2	613	1	6	
1467	27	0	2	155	1	4	
1468	49	0	1	1023	2	2	
1469	34	0	2	628	1	8	

1470 rows × 34 columns

In [90]:
```
# Normalize the data
scaler = StandardScaler()
numeric_cols = df.select_dtypes(include=[np.number]).columns
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
df
```

Out[90]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHo
0	0.446350	2.280906	0.590048	0.742527	1.401512	-1.0109
1	1.322365	-0.438422	-0.913194	-1.297775	-0.493817	-0.1471
2	0.008343	2.280906	0.590048	1.414363	-0.493817	-0.8875
3	-0.429664	-0.438422	-0.913194	1.461466	-0.493817	-0.764
4	-1.086676	-0.438422	0.590048	-0.524295	-0.493817	-0.8875
...	...	...	...	...	...	
1465	-0.101159	-0.438422	-0.913194	0.202082	-0.493817	1.7037
1466	0.227347	-0.438422	0.590048	-0.469754	-0.493817	-0.3939
1467	-1.086676	-0.438422	0.590048	-1.605183	-0.493817	-0.6407
1468	1.322365	-0.438422	-0.913194	0.546677	1.401512	-0.8875
1469	-0.320163	-0.438422	0.590048	-0.432568	-0.493817	-0.1471

1470 rows × 34 columns

In [91]:
```
#import more libraries
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics import confusion_matrix, accuracy_score, precision_scor
```

In [135…

```
#Machine learning model
#logistic regression


# Features
X = df[["Age","MonthlyIncome","YearsAtCompany",
        "JobSatisfaction","WorkLifeBalance"]]

# Categorical target (0 = No, 1 = Yes)
y = (df["Attrition"] > 0).astype(int)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Predictions
predictions = model.predict(X_test)
print(predictions)

# Attrition:
# 0 = No (stayed)
# 1 = Yes (left)

#evaluating logistic regression model
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(y_test, predictions))
print("Recall:", recall_score(y_test, predictions))
print("F1 Score:", f1_score(y_test, predictions))
print("Detailed Classification Report:")
print(classification_report(y_test, predictions))
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

for i in range(3):
    subset = df[df['Cluster'] == i]
    plt.scatter(subset['MonthlyIncome'],
                subset['YearsAtCompany'],
                label=f'Cluster {i}')

plt.xlabel('Monthly Income')
plt.ylabel('Years at Company')
plt.title('Employee Clusters')
plt.legend()
plt.show()
```



In [97]:
```
#average values in each cluster
df.groupby('Cluster')[['Age','MonthlyIncome','YearsAtCompany',
                       'JobSatisfaction','WorkLifeBalance']].mean()
```

Out[97]:

Cluster	Age	MonthlyIncome	YearsAtCompany	JobSatisfaction	WorkLifeBalanc
0	-0.263297	-0.367930	-0.276737	-1.133845	0.01297
1	-0.235862	-0.372129	-0.250319	0.719870	0.00078
2	1.141540	1.716274	1.206690	-0.024873	-0.02538

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: